FULL-STACK / ACADEMIC PROJECT

Vehicle Rental Management System

A web application that connects a premium rental experience to reliable Java business logic.

REACT + SPRING BOOT + MYSQL



01 / FRONTEND • BACKEND • DATABASE

Designed for travel. Built on Java.

JAVA / OBJECT-ORIENTED PROGRAMMING

One platform. Two perspectives.

The system connects people looking for vehicles with the team managing the rental fleet.

CUSTOMER EXPERIENCE

Find, choose, rent.

Browse available vehicles
Check dates & rates
View rental history

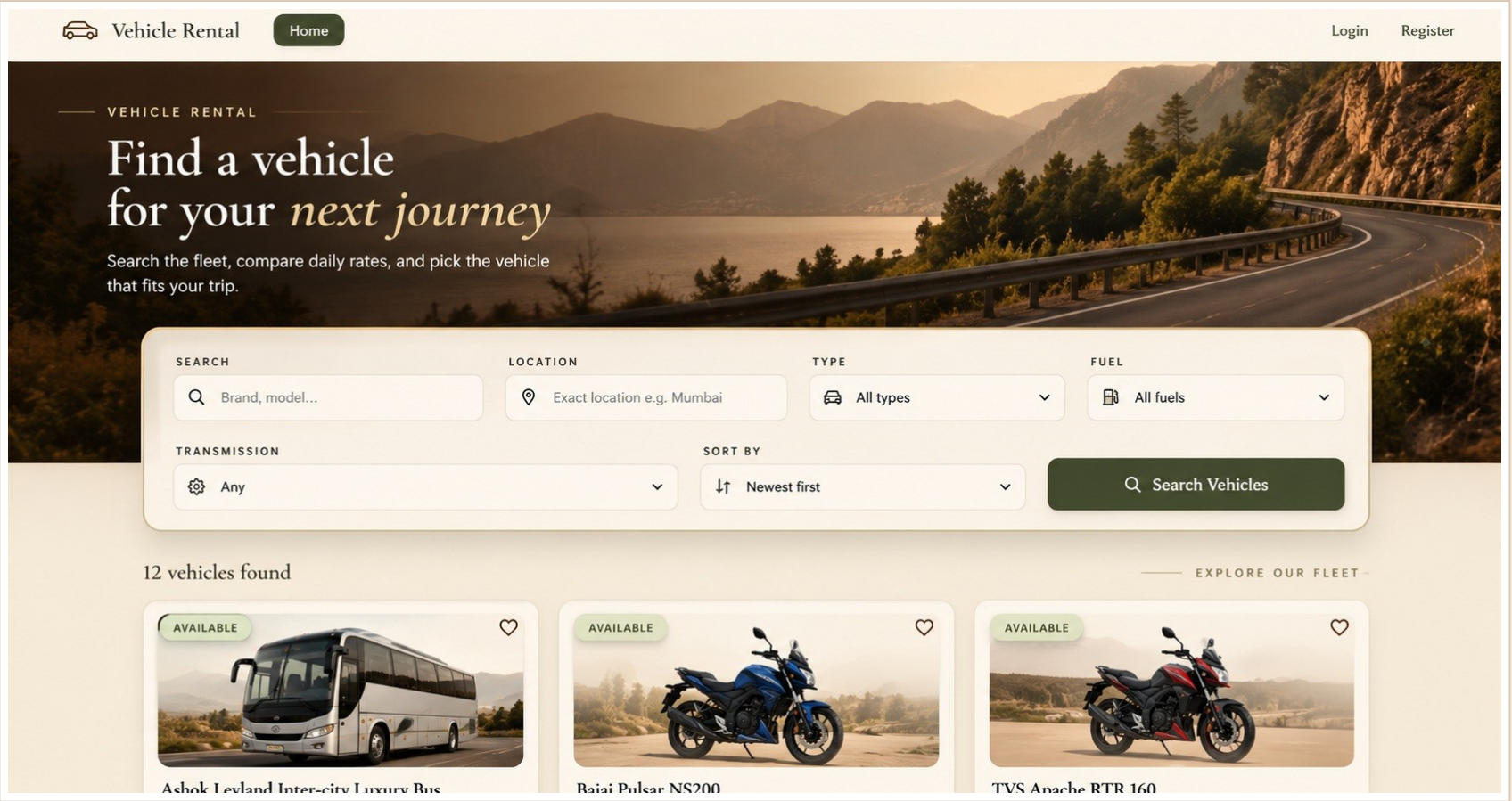
ADMIN EXPERIENCE

Control the operation.

Manage vehicles & customers
Track rentals and payments
Maintain fleet availability

CORE GOAL Reduce manual coordination while keeping bookings and records consistent.

The journey begins on the website.



LANDING-PAGE DESIGN REFERENCE • REACT + VITE

FRONTEND FLOW

01 **Discover the fleet**

Vehicles with photos & rates

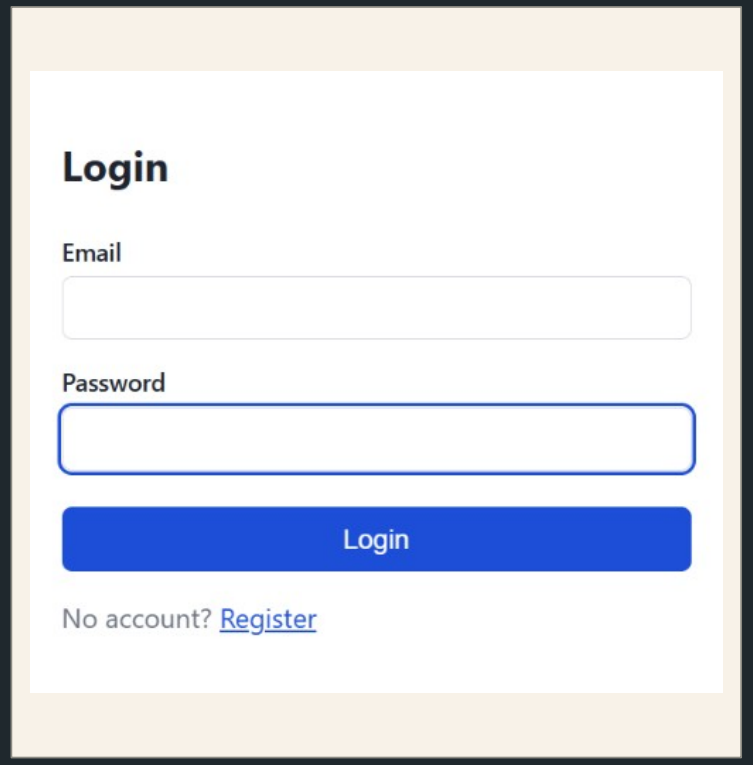
02 **Refine the search**

Type, fuel, location & sort

03 **Inspect details**

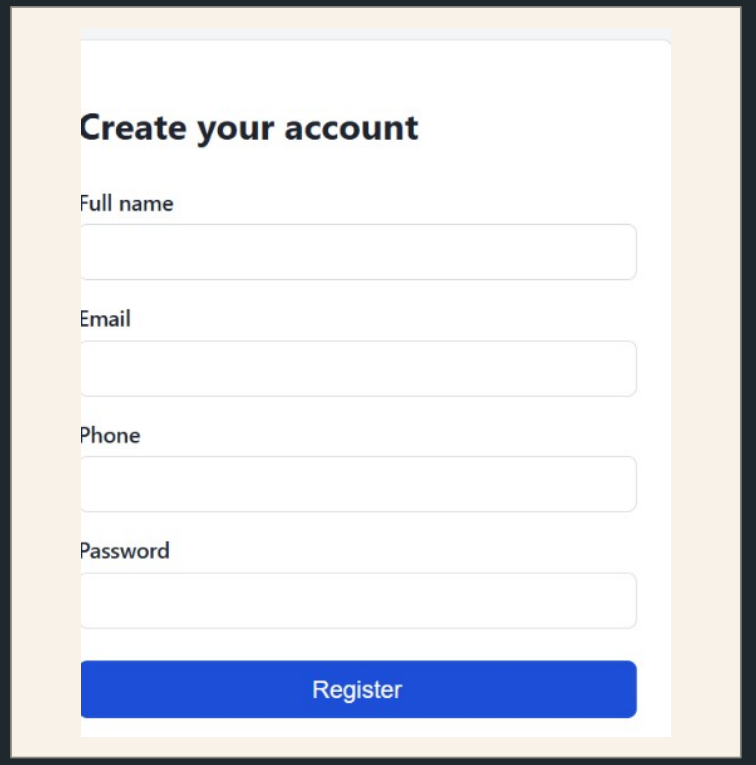
Availability before booking

The screens behind the experience.



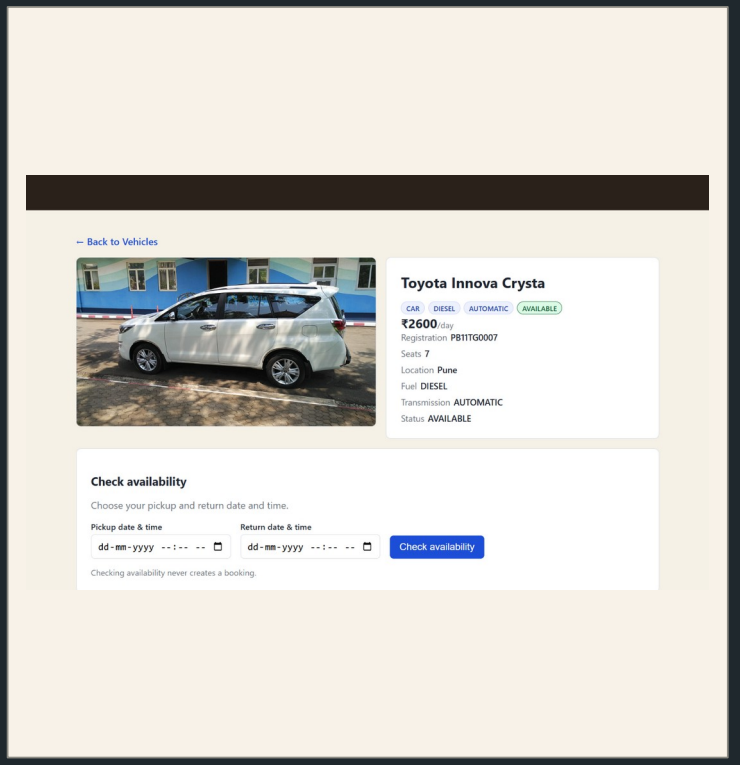
01 Sign in

Email and password



02 Create an account

Customer registration



03 Vehicle details

Rates and availability

From a click to a rental record.

ONE BOOKING, FOUR DECISIONS

01 — Choose a vehicle

Open its details & rate

02 — Select rental dates

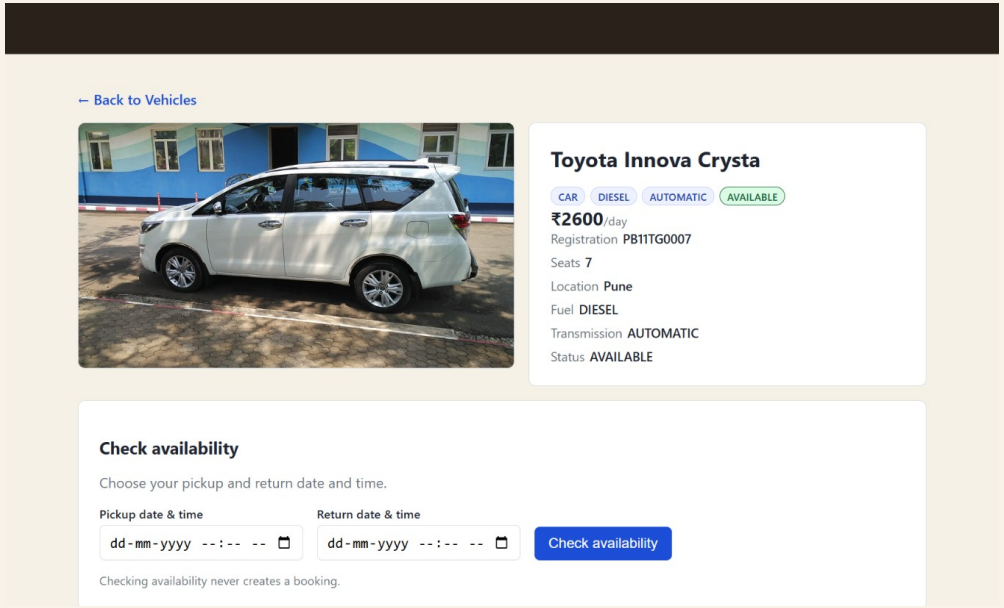
Pickup must be in the future

03 — Validate availability

Reject overlapping bookings

04 — Create rental

Store price & status



VEHICLE DETAILS / AVAILABILITY

BUSINESS RULES: date validation / conflict check / price breakdown

Three parts. One connected system.

01 / USER LAYER

React + Vite

What the user sees

Search, forms, pages
Customer & admin UI

→

02 / LOGIC LAYER

Spring Boot

What Java decides

REST controllers
Services, validation, JWT

→

03 / DATA LAYER

MySQL

What the system remembers

Vehicles, users, rentals
Payments & maintenance

CUSTOMER ACTION → API REQUEST → JAVA VALIDATION → DATABASE → RESPONSE

Where the Java backend begins.

SPRING BOOT / REQUEST PATH

CONTROLLER	Accepts HTTP requests
↓	
SERVICE	**Applies rental logic**
↓	
REPOSITORY	Reads and writes entities
↓	
MYSQL	Persists related records

VehicleRentalApplication.java — JAVA SOURCE

```java
package com.vehicle.rental;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class VehicleRentalApplication {

    public static void main(String[] args) {
        SpringApplication.run(VehicleRentalApplication.class, args);
    }
}
```

ENTRY POINT	main() starts the Spring Boot application
SCHEDULING	@EnableScheduling enables scheduled tasks

CORE ENTITIES User Vehicle Rental Payment Maintenance

Encapsulation: protect the Rental state

Private fields + public methods = controlled access to an object.

Rental.java (concept example)

JAVA / EDITOR VIEW

```java
public class Rental {
    private RentalStatus status;
    private BigDecimal totalAmount;

    public RentalStatus getStatus() {
        return status;
    }

    public void updateStatus(RentalStatus next) {
        if (next == null) throw new IllegalArgumentException();
        this.status = next;
    }
}
```

Java | UTF-8 | Project-domain illustration

PRIVATE DATA

The status and total amount stay inside the class.

CONTROLLED ACCESS

Other classes interact through methods, not fields.

ILLUSTRATIVE JAVA / NOT A VERIFIED FILE SCREENSHOT

Interfaces & abstraction in Spring Data

A repository contract separates business logic from the way records are stored.

VehicleRepository.java / RentalService.java (concept example)

JAVA / EDITOR VIEW

```java
public interface VehicleRepository
        extends JpaRepository<Vehicle, Long> {
}

public class RentalService {
    private final VehicleRepository vehicles;
    public RentalService(VehicleRepository vehicles) {
        this.vehicles = vehicles;
    }
    public Optional<Vehicle> findVehicle(long id) {
        return vehicles.findById(id);
    }
}
```

Java | UTF-8 | Project-domain illustration

INTERFACE

VehicleRepository defines an agreed way to access vehicles.

HIDDEN IMPLEMENTATION

findById() is called without writing the SQL in the service.

ILLUSTRATIVE JAVA / NOT A VERIFIED FILE SCREENSHOT

Polymorphism: code to an interface

The same repository type works with Spring Data’s runtime implementation.

VehicleService.java (concept example)

JAVA / EDITOR VIEW

```java
public class VehicleService {
    private final VehicleRepository vehicles;

    public VehicleService(VehicleRepository vehicles) {
        this.vehicles = vehicles;
    }

    public Vehicle load(long id) {
        return vehicles.findById(id).orElseThrow();
    }
}
```

Java | UTF-8 | Project-domain illustration

INTERFACE REFERENCE

The service depends on VehicleRepository, not a concrete database class.

RUNTIME DISPATCH

Spring supplies an implementation for calls such as findById().

ILLUSTRATIVE JAVA / NOT A VERIFIED FILE SCREENSHOT